

DataShare — Support de Présentation

Auteur : Mathieu CHEREAU
Projet : P3 — Pilotez le développement d'une solution informatique
Date : Juin 2026

Slide 1 — Contexte & Problème

DataShare — Plateforme de transfert sécurisé de fichiers

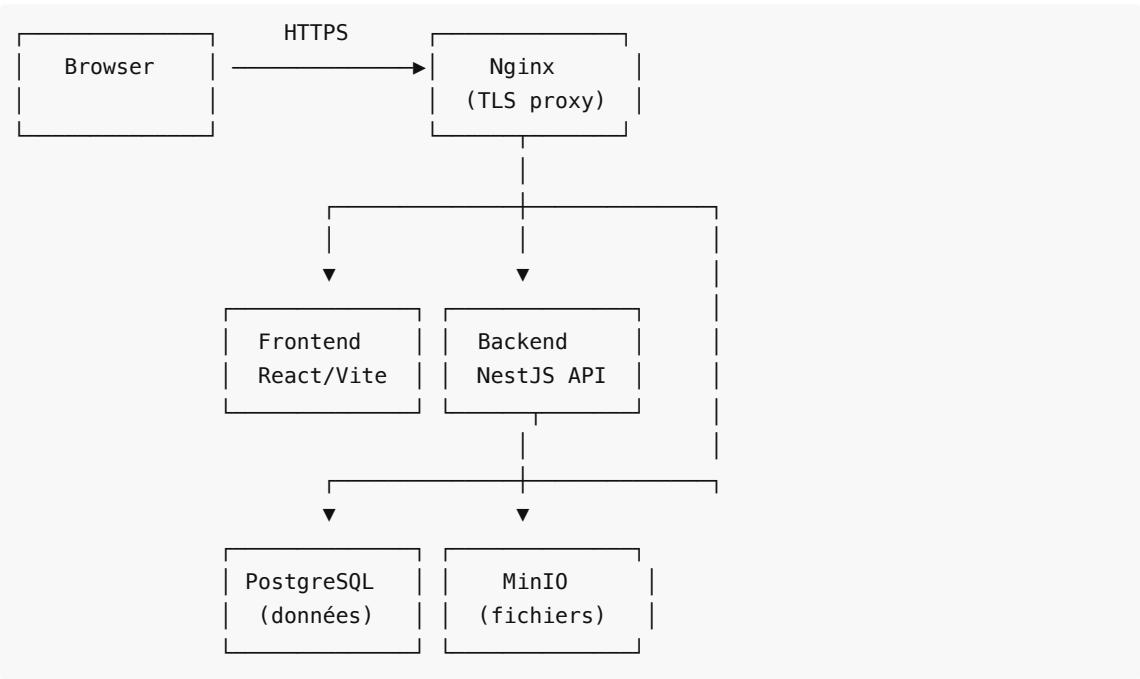
Contexte : Les freelances et petites entreprises ont besoin d'échanger des fichiers de manière sécurisée, sans dépendre de services cloud tiers coûteux.

Problème résolu :

- Upload de fichiers avec stockage sécurisé
- Liens de téléchargement temporaires et protégeables par mot de passe
- Gestion autonome des fichiers (liste, suppression, tags)
- Authentification sécurisée (JWT + refresh tokens)

Objectif MVP : Prototype fonctionnel en 4 semaines pour une démo investisseurs.

Slide 2 — Architecture technique



5 services conteneurisés avec Docker Compose — démarrage en une commande (`make up`).

Slide 3 — Stack technique & justifications

Couche	Technologie	Justification
--------	-------------	---------------

Frontend	React 18 + Vite + TypeScript	Écosystème mature, HMR rapide, typage statique
Backend	NestJS 10 + TypeScript	Architecture modulaire, injection de dépendances
BDD	PostgreSQL 16 + Prisma ORM	Robuste, ORM type-safe, migrations auto
Stockage	MinIO (S3-compatible)	Self-hosted, compatible AWS S3, zéro coût
Auth	JWT + Refresh Tokens	Stateless, sécurisé, révocable
Proxy	Nginx	TLS termination, routing, headers sécu
Tests	Jest + Playwright + k6	Unitaires + E2E + charge

Slide 4 — Modèle de données

4 entités principales :

Entité	Description	Relations
User	Utilisateur inscrit	1:N → File
File	Fichier uploadé + stocké MinIO	N:1 → User, 1:N → DownloadLink, M:N → Tag
DownloadLink	Lien temporaire sécurisé	N:1 → File
Tag	Étiquette d'organisation	M:N → File

Points clés :

- UUIDs comme clés primaires
- Mots de passe fichiers hashés bcrypt
- Suppression en cascade (File → DownloadLink)

Slide 5 — Fonctionnalités implémentées

Authentification (US03 + US04)

-  Inscription avec validation email/mot de passe
-  Connexion avec JWT (access 15min + refresh 7j)
-  Logout avec révocation du refresh token
-  Refresh automatique du token

Gestion de fichiers (US01 + US05)

-  Upload multipart (max 50MB)
-  Liste paginée avec recherche et filtrage par tags
-  Métadonnées (taille, type MIME, date)
-  Suppression avec nettoyage MinIO

Liens de téléchargement (US02 + US07)

-  Génération de liens temporaires (1h / 24h / 7j)
-  Protection par mot de passe (optionnel)
-  Compteur de téléchargements

-  Expiration automatique

Fonctionnalités avancées (US08 + US09)

-  Upload anonyme (sans compte)
-  Système de tags sur les fichiers

Slide 6 — Sécurité

Mesure	Implémentation
Chiffrement en transit	HTTPS/TLS via Nginx
Hachage mots de passe	bcrypt (10 rounds)
Tokens signés	JWT HS256, access 15min, refresh 7j
Refresh token hashé	Stocké hashé en BDD, révocable
Liens sécurisés	Token UUID unique + TTL
Headers sécu	X-Frame-Options, HSTS, nosniff
Secrets externalisés	Variables d'env (.env non versionné)
Fail-fast	L'app refuse de démarrer si secret manquant

Slide 7 — Qualité & Tests

Couverture de tests

Type	Outil	Nombre	Couverture
Unitaires	Jest	40+ tests	72.82% statements
E2E	Playwright	10 specs	10 user stories
Charge	k6	3 scénarios	Upload concurrent

Qualité de code

- **Conventional Commits** : historique structuré
- **Pull Requests** : review avant merge, squash merge
- **Linting** : ESLint + Prettier
- **GitHub Issues** : traçabilité feature → issue → PR

Slide 8 — Gestion de projet

Méthodologie

- **Trunk-based development** : branche `main` comme source de vérité
- **Feature branches** : `feat/*` , `fix/*` , `chore/*`
- **Issues GitHub** : chaque fonctionnalité tracée
- **Conventional Commits** : `feat:` , `fix:` , `docs:` , `test:` , `chore:`

Utilisation de l'IA

- **GitHub Copilot** : autocomplétion supervisée
 - **Claude (Anthropic)** : architecture, documentation, refactoring
 - **Supervision systématique** : chaque sortie IA relue, testée, adaptée
 - **Gain estimé** : 40-50% sur le temps de développement
-

Slide 9 — Démo

Parcours démontré :

1. **Inscription** → Création de compte (email + mot de passe)
2. **Connexion** → Authentification JWT
3. **Upload** → Sélection de fichier + mot de passe optionnel + expiration
4. **Lien généré** → URL de téléchargement sécurisée
5. **Téléchargement** → Via le lien (avec mot de passe si défini)
6. **Dashboard** → Liste des fichiers, suppression, tags

Accès :

```
https://localhost      → Interface utilisateur
https://localhost/api   → API REST
http://localhost:9001   → Console MinIO
```

Slide 10 — Perspectives & Conclusion

Évolutions futures

-  Notifications email (liens partagés)
-  Analytics (statistiques de téléchargement détaillées)
-  Chiffrement at-rest (fichiers chiffrés sur MinIO)
-  Migration cloud (AWS S3 / Azure Blob)
-  Espaces de travail collaboratifs

Points forts du MVP

-  Architecture propre et maintenable
-  Sécurité dès le départ (HTTPS, JWT, bcrypt, headers)
-  Tests multi-couches (unitaires + E2E + charge)
-  Déploiement en une commande (`make up`)
-  Documentation technique complète
-  Historique Git structuré avec traçabilité

Conclusion

DataShare est prêt pour la démo investisseurs : un MVP fonctionnel, sécurisé et bien documenté, avec une base de code solide pour les évolutions futures.

Merci pour votre attention !

Mathieu CHEREAU — Juin 2026